Random Forest, Boosting, AdaBoost

Simon Liu

Oct 22, 2026

Review

Variance Reduction for Correlated Models (Part 1)

Regression Setup

- Suppose we have M models, each predicting:

$$\hat{f}_m(x) = f(x) + \epsilon_m$$

- Where:
  - $f(x)$ = True underlying function.
  - ϵ_m = Zero-mean noise with variance σ^2.
- Noise terms are no longer independent — they now have **correlation**.

Averaged Model Prediction

- The ensemble's prediction is:

$$\hat{f}_{\text{avg}}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x)$$

- Variance of the averaged model:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \text{Var}\left(\frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x) \right)$$

Variance Reduction for Correlated Models (Part 2)

Step 1: Definition of Variance

- By definition,

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \mathbb{E}\left[\left(\hat{f}_{\text{avg}}(x) - \mathbb{E}[\hat{f}_{\text{avg}}(x)]\right)^2\right]$$

Step 2: Finding $\mathbb{E}[\hat{f}_{\text{avg}}(x)]$

- From our earlier setup,

$$\hat{f}_m(x) = f(x) + \epsilon_m$$

- Taking the expectation:

$$\mathbb{E}[\hat{f}_m(x)] = \mathbb{E}[f(x) + \epsilon_m]$$

- Since $f(x)$ is deterministic and $\mathbb{E}[\epsilon_m] = 0$,

$$\mathbb{E}[\hat{f}_m(x)] = f(x)$$

- Therefore:

$$\mathbb{E}[\hat{f}_{\text{avg}}(x)] = f(x)$$

Variance Reduction for Correlated Models (Part 3)

Step 3: Expanding the Variance Definition

- Now return to the variance formula:

$$\text{Var}(\hat{f}_{\text{avg}}(x)) = \mathbb{E}\left[\left(\frac{1}{M}\sum_{m=1}^{M}\hat{f}_m(x) - f(x)\right)^2\right]$$

- Since $f(x)$ is constant:

$$= \mathbb{E}\left[\left(\frac{1}{M}\sum_{m=1}^{M}\epsilon_m\right)^2\right]$$

- Expanding the square:

$$= \mathbb{E}\left[\frac{1}{M^2}\sum_{m=1}^{M}\epsilon_m^2 + \frac{2}{M^2}\sum_{1\le i<j\le M}\epsilon_i\epsilon_j\right]$$

- Taking expectations:

$$= \frac{1}{M^2}\sum_{m=1}^{M}\sigma^2 + \frac{2}{M^2}\sum_{1\le i<j\le M}\rho\sigma^2$$

Variance Reduction for Correlated Models (Part 4)

Step 4: Simplifying the Terms

- First term:

$$\frac{1}{M^2}\sum_{m=1}^{M}\sigma^2 = \frac{\sigma^2}{M}$$

- Second term:

$$\frac{2}{M^2}\cdot\frac{M(M-1)}{2}\cdot\rho\sigma^2 = \frac{M(M-1)}{M^2}\rho\sigma^2$$

- Combining both:

$$\mathrm{Var}(\hat{f}_{\mathrm{avg}}(x)) = \sigma^2\left(\frac{1-\rho}{M}+\rho\right)$$

Key Observations:

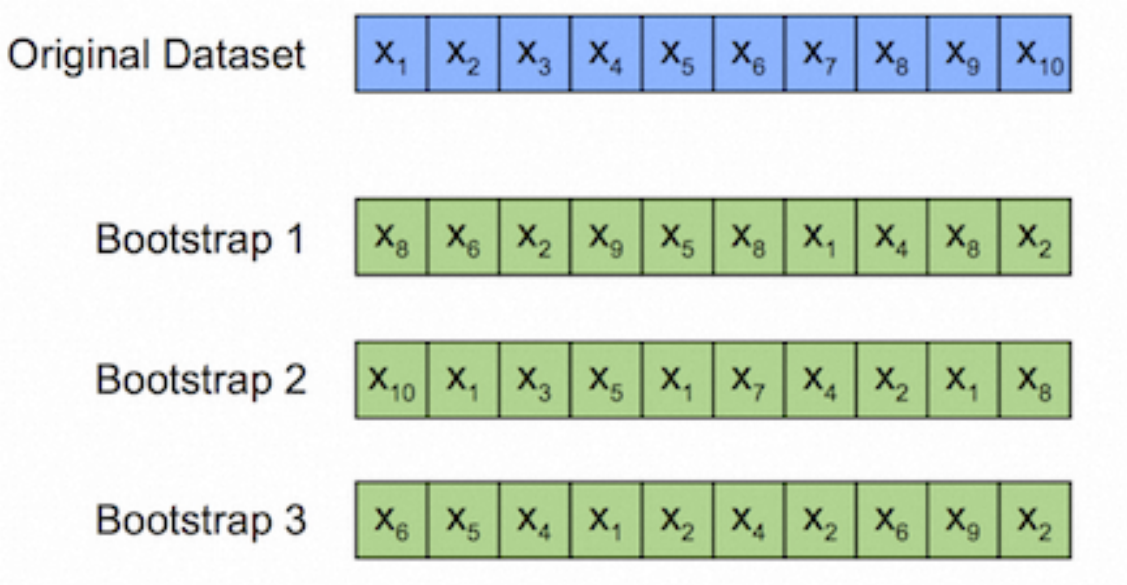
- If models are **uncorrelated** ($\rho = 0$), variance reduces by $\frac{1}{M}$.
- If models are **fully correlated** ($\rho = 1$), averaging provides **no variance reduction**.
- Effective ensemble methods aim to reduce ρ through techniques like **bootstrapping** and **Random Forests**.

Review of Bootstrapping

Bootstrapping is a resampling technique that generates multiple datasets by sampling with replacement from the original dataset.

Setup:

- Given a dataset $D = \{x_1, x_2, \ldots, x_n\}$ of size n.
- Create M bootstrap samples: $D_1, D_2, \ldots, D_M$.
- Each bootstrap sample is drawn by sampling n points **with replacement** from the original dataset.
- Some points may appear multiple times; some may not appear at all.



This work by Sebastian Raschka is licensed under a Creative Commons Attribution 4.0 International License.

Bootstrapping Theory: Expected Coverage of Original Dataset

Proportion of Unique Points in a Bootstrap Sample:

- Probability that a given point is **not selected** in a single draw: $1 - \frac{1}{n}$
- Probability that a point is **never selected** in n draws (full bootstrap sample): $\left(1 - \frac{1}{n}\right)^n$
- As $n \to \infty$, this converges to: $\lim_{n\to\infty} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e} \approx 0.368$

Conclusion: In large datasets, approximately 63% of the original data points appear in a typical bootstrap sample, while 37% are left out.

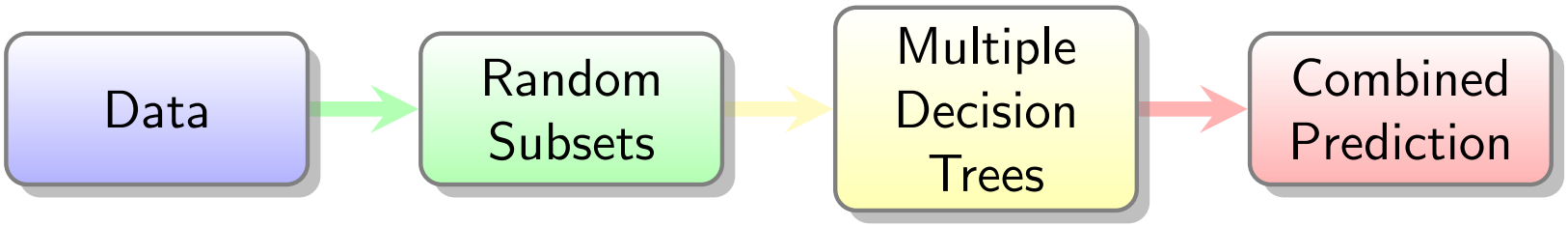
Random Forest

Introduction to Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and control overfitting.

Key Benefits:

- Handles large datasets effectively
- Reduces overfitting through averaging or majority voting
- Requires minimal tuning compared to single decision trees



Building Bootstrap Samples in Random Forest

Bootstrap Sampling in Random Forest:

- Random Forest builds M decision trees, each trained on a unique bootstrap sample.
- Each bootstrap sample is generated by sampling n points with replacement from the original dataset.

Why Bootstrap Sampling?

- Introduces randomness, ensuring diversity across trees.
- Reduces variance through aggregation, improving model robustness.

Tree Growth in Random Forest

Feature Randomization During Tree Growth:

- At each split in the tree, a **random subset of features** (of size $k < p$) is selected.
- The best split is chosen only from this subset.
- Typical values:
  - $k = \sqrt{p}$
  - $k = \log p$

Purpose of Feature Randomization:

- Reduces correlation between trees.
- Increases model diversity and robustness.
- Ensures no single strong predictor dominates the entire forest.

Random Forest Prediction

Prediction in Random Forest:
For Regression:

- Each tree predicts a numeric value.
- The final prediction is the **average** of all tree predictions:

$$\hat{f}_{RF}(x) = \frac{1}{M} \sum_{m=1}^{M} T_m(x)$$

For Classification:

- Each tree predicts a class label.
- The final prediction is determined by **majority voting** across all trees:

$$\hat{f}_{RF}(x) = \text{mode}\{T_1(x), T_2(x), \ldots, T_M(x)\}$$

Variance Reduction in Random Forest

Variance Analysis:

- Random Forest relies on deep decision trees, which are naturally **high variance, low bias** models.
- Aggregating multiple deep trees helps reduce variance.

Variance Formula:

$$\text{Var}(\hat{f}_{RF}(x)) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$$

Explanation:

- $\sigma^2 =$ Variance of a single tree.
- $\rho =$ Correlation between trees.
- $M =$ Number of trees.

Key Insight: Increasing M reduces the second term, improving stability. Lowering correlation between trees (via feature randomization) further enhances variance reduction.

Introduction to Feature Importance

What is Feature Importance?

- Feature importance quantifies how much each feature contributes to the model's predictive power.
- Useful for:
  - Identifying key features driving predictions.
  - Improving model interpretability.
  - Reducing dimensionality by removing less important features.

Example Use Cases:

- Medical diagnosis — Identifying the most important symptoms.
- E-commerce — Identifying key customer behaviors for product recommendations.

Impurity-Based Feature Importance

Definition: Impurity-based feature importance measures the contribution of each feature to impurity reduction across all splits in the forest.
Calculation: For each feature X_j:

$$\text{Importance}(X_j) = \sum_{t=1}^{M} \sum_{s \in \text{splits on } X_j} \Delta\text{Impurity}(s, t)$$

Where:

- $\Delta\text{Impurity}(s, t)$ = Reduction in impurity at split s in tree t.
- M = Number of trees in the forest.

Note: Features that reduce impurity significantly will score higher.

Limitations of Impurity-Based Importance

Bias Towards High-Cardinality Features:

- Features with many unique values (e.g., IDs, dates, etc.) tend to score higher, even if they are less meaningful.
- This happens because high-cardinality features create more potential splits, which may reduce impurity even without adding meaningful predictive power.

Example:

- A random ID column may rank highly despite being irrelevant.

Takeaway: Impurity-based feature importance may incorrectly emphasize features with many distinct values.

Permutation Feature Importance

Definition: Permutation importance measures a feature's importance by evaluating the change in model performance after randomly shuffling its values.

Process:

1. Train the Random Forest model on original data.
2. For each feature:
   - Shuffle the feature's values.
   - Measure the change in model performance.
3. The larger the performance drop, the more important the feature is.

Key Idea: Important features have a significant impact on predictions when their values are randomized.

Benefits and Technicalities of Permutation Importance

Benefits:

- Unbiased toward high-cardinality features.
- Provides a robust measure of feature importance in correlated data.

Technical Considerations:

- Permutation importance is calculated **after training**, ensuring it reflects true feature influence.
- The technique requires repeated evaluations, which may increase computational cost.

Comparing Impurity-Based and Permutation Importance

Aspect	**Impurity Importance**	**Permutation Importance**
Default in scikit-learn	Yes	No
Calculation Timing	During Training	After Training
Bias	Biased toward high-cardinality	Unbiased
Interpretation	Based on impurity reduction	Based on prediction impact
Robustness	May overestimate importance	More reliable in complex data
Computational Cost	Lower (calculated during training)	Higher (requires repeated evaluations)

Out-of-Bag (OOB) Concept

Idea:

- During bootstrap sampling, about 37% of the original data is left out.
- These unused points are called **Out-of-Bag (OOB)** samples.
- Each tree in the Random Forest can be evaluated on its corresponding OOB data to assess model performance.

Visual Representation:



This work by Sebastian Raschka is licensed under a Creative Commons Attribution 4.0 International License.

OOB Error Calculation (Technical Details)

OOB Error Calculation:

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \hat{f}_{\text{OOB}}(x_i))$$

Where:

- y_i = True label of data point i.
- $\hat{f}_{\text{OOB}}(x_i)$ = Average prediction from trees where x_i was **not included** in training.
- L = Loss function (e.g., mean squared error for regression, log loss for classification).

Key Benefit: OOB error provides a built-in cross-validation measure, reducing the need for a separate validation set.

Practical Implementation in Random Forest

- **Number of Trees:** Controls the number of trees in the forest. Increasing this improves stability but adds computational cost.
  - Default: `n_estimators=100`
- **Number of Features at Each Split:** Specifies the number of random features considered at each node split.
  - Default: `max_features='sqrt'` for classification
- **Max Depth:** Limits the maximum depth of each tree to control model complexity.
  - Default: `max_depth=None` (nodes are expanded until all leaves are pure).
- **Min Samples Split:** Minimum number of samples required to split an internal node. Larger values reduce model complexity.
  - Default: `min_samples_split=2`
- **Min Samples Leaf:** Minimum number of samples required at a leaf node. Ensures leaves contain sufficient data for stable predictions.
  - Default: `min_samples_leaf=1`

Benefits of Random Forest

Key Benefits:

- **Feature Importance:** Provides a clear measure of each feature's contribution to prediction accuracy, aiding in feature selection and interpretability.
- **Handling Missing Values:** Random Forest can naturally handle missing data by splitting on surrogate variables when primary splitting variables are missing.
- **Parallelization:** Each tree is grown independently, making Random Forest highly parallelizable and efficient on modern hardware.
- **Robustness to Noise and Outliers:** Due to averaging over multiple trees, Random Forest is less prone to overfitting and more stable when handling noisy data.
- **Versatility:** Effective for both classification and regression tasks without the need for extensive parameter tuning.

Coding Sample Part 1

Break

What is Boosting?

Definition:

- Boosting is an **ensemble learning technique** that combines multiple weak learners to form a stronger predictive model.
- Each weak learner corrects the mistakes of the previous one, emphasizing harder-to-classify points.

Key Idea: Sequential learning with each model improving upon the previous.

Why "Boosting"?

- The term "boosting" refers to the idea of **boosting** the performance of weak models by sequentially enhancing their accuracy.

Example Analogy: Think of boosting like refining an essay: successive reviews correct the remaining mistakes until the essay is polished.

Advantages and Challenges of Boosting

Advantages:

- Often achieves **high accuracy**.
- Effective for **complex data patterns** and **imbalanced data**.
- Provides **flexibility** to use various weak learners.

Challenges:

- Can **overfit** without proper tuning.
- Tends to be **computationally intensive**.
- Requires more **careful hyperparameter tuning**.

Initialization

Initialization:

- Given training data: $(x_1, y_1), \ldots, (x_n, y_n)$
- Assign initial weights to each sample:

$$w_{i1} = \frac{1}{n}, \quad \text{for all } i = 1, 2, \ldots, n$$

General Boosting Algorithm - Iterative Learning Process

For $m = 1, 2, \ldots, M$**:**

1. **Fit a Weak Learner:** Train a weak learner $h_m(x)$ to minimize the weighted error:

$$h_m = \arg\min_h \sum_{i=1}^{n} w_{i,m} \mathbb{I}(y_i \neq h(x_i))$$

2. **Compute Error:** Compute the weighted error for the weak learner:

$$\text{err}_m = \sum_{i=1}^{n} w_{i,m} \mathbb{I}(y_i \neq h_m(x_i))$$

3. **Update Weights:** Increase the weights of misclassified points:

$$w_{i,m+1} = w_{im} \times \text{Adjustment Factor}$$

4. **Normalize Weights:**

$$w_{i,m+1} = \frac{w_{i,m+1}}{\sum_{j=1}^{n} w_{j,m+1}}$$

General Boosting Algorithm - Final Model

Final Model:

- After M steps, the boosted model is:

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

- Where:
  - $h_m(x)$ = Weak learner at step m
  - α_m = Weight assigned to $h_m(x)$

Key Insight:

- The model iteratively reduces errors by re-weighting observations to focus on difficult cases.
- Boosting excels by combining many weak learners to create a powerful ensemble predictor.

Introduction to Adaboost

Historical Context:

- **Adaboost** (Adaptive Boosting) was introduced by **Freund and Schapire** in **1996**.
- It was the first practical boosting algorithm with strong theoretical foundations.
- Adaboost's development was pivotal in proving that weak learners can be combined to create a strong classifier.

Basic Idea:

- Adaboost minimizes the **exponential loss**:

$$L(y, F(x)) = e^{-yF(x)}$$

- Labels must be encoded as $y \in \{-1, +1\}$ to align with the exponential loss formulation.

Understanding the Exponential Loss

Exponential Loss Function:

$$L(y, F(x)) = e^{-yF(x)}$$

Why the Exponential Loss?

- Exponential loss heavily penalizes misclassified points.
- Aligns with AdaBoost's strategy of focusing on hard-to-classify points.
- Minimizing this loss iteratively improves model performance.

Decomposing the Exponential Loss

Step 1: Assume the current model is $F_{m-1}(x)$ and the next weak learner is $h_m(x)$.

$$F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$$

Step 2: The exponential loss for this new model is:

$$\sum_{i=1}^{n} e^{-y_i F_m(x_i)} = \sum_{i=1}^{n} e^{-y_i (F_{m-1}(x_i) + \alpha_m h_m(x_i))}$$

Step 3: Expanding the exponential term:

$$\sum_{i=1}^{n} e^{-y_i F_{m-1}(x_i)} e^{-\alpha_m y_i h_m(x_i)}$$

Since $e^{-y_i F_{m-1}(x_i)}$ does not depend on α_m, minimizing the exponential loss is equivalent to minimizing:

$$\sum_{i=1}^{n} w_{im} e^{-\alpha_m y_i h_m(x_i)}$$

Where $w_{im} = e^{-y_i F_{m-1}(x_i)}$ are the current sample weights.

Minimizing the Weighted Loss

Step 1: Split the weighted loss into correctly and incorrectly classified points:

$$\sum_{i=1}^{n} w_{im} e^{-\alpha_m y_i h_m(x_i)} = \sum_{y_i = h_m(x_i)} w_{im} e^{-\alpha_m} + \sum_{y_i \neq h_m(x_i)} w_{im} e^{\alpha_m}$$

Step 2: Define the total error:

$$\text{err}_m = \sum_{y_i \neq h_m(x_i)} w_{im}$$

Step 3: Substitute this back into the weighted loss:

$$\sum_{i=1}^{n} w_{im} e^{-\alpha_m y_i h_m(x_i)} = (1 - \text{err}_m) e^{-\alpha_m} + \text{err}_m e^{\alpha_m}$$

Deriving the Optimal α_m

Step 1: Minimize the expression:

$$\min_{\alpha_m}(1 - \text{err}_m)e^{-\alpha_m} + \text{err}_m e^{\alpha_m}$$

Step 2: Differentiate with respect to α_m:

$$\frac{\partial}{\partial \alpha_m}[(1 - \text{err}_m)e^{-\alpha_m} + \text{err}_m e^{\alpha_m}] = 0$$

Step 3: Solving this gives:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$$

Why Does the Exponential Weight Update Work?

Key Insight:

$$w_{i,m+1} = w_{im} e^{-\alpha_m y_i h_m(x_i)}$$

Why This Works:

- Correctly classified points (where $y_i = h_m(x_i)$) are multiplied by $e^{-\alpha_m}$, reducing their influence.
- Misclassified points (where $y_i \neq h_m(x_i)$) are multiplied by e^{α_m}, increasing their influence.

Result: The exponential function heavily penalizes errors, ensuring the model prioritizes difficult points in the next iteration.

Conclusion on α_m

Why is α_m Important?

- α_m controls how much influence each weak learner has in the final model.
- If err_m is small (meaning the weak learner performs well), α_m will be large — giving this learner more influence.
- Conversely, if err_m is close to 0.5 (random guessing), α_m will be close to zero — reducing the learner's impact.

Key Insight: The log-formula for α_m ensures AdaBoost carefully balances weak learners' influence based on their accuracy.

Adaboost Algorithm - Initialization

Step 1: Initialization

- Given training data: $(x_1, y_1), \ldots, (x_n, y_n)$
- Initialize the weights for each sample:

$$w_{i1} = \frac{1}{n}, \quad \text{for all } i = 1, 2, \ldots, n$$

Adaboost Algorithm - Iterative Process

Step 2: Iterative Learning Process

- For $m = 1, 2, \ldots, M$:
  1. **Fit a Weak Learner:** Train a weak learner $h_m(x)$ to minimize the weighted error:

$$h_m = \arg\min_h \sum_{i=1}^{n} w_{i,m} \mathbb{I}(y_i \neq h(x_i))$$

  2. **Compute Model Weight:**

$$\alpha_m = \frac{1}{2} \ln\left(\frac{1 - \text{err}_m}{\text{err}_m}\right)$$

  where $\text{err}_m = \sum_{i=1}^{n} w_{im} \mathbb{I}(y_i \neq h_m(x_i))$

  3. **Update Sample Weights:**

$$w_{i,m+1} = w_{im} e^{-\alpha_m y_i h_m(x_i)}$$

  4. **Normalize Weights:**

$$w_{i,m+1} = \frac{w_{i,m+1}}{\sum_{j=1}^{n} w_{j,m+1}}$$

Adaboost Algorithm - Final Model

Step 3: Final Model Construction

- After M steps, the boosted model is:

$$F(x) = \text{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(x)\right)$$

- Where:
  - $h_m(x)$ = Weak learner at step m
  - α_m = Weight assigned to $h_m(x)$

Key Insight:

- The model iteratively reduces errors by re-weighting observations to focus on difficult cases.
- The exponential loss ensures that misclassified points receive greater emphasis in subsequent iterations.

Choosing Weak Learners for AdaBoost

Key Principles for Weak Learner Selection:

- **High Bias Models:** Boosting reduces bias over iterations, so starting with models that have **high bias** is acceptable.
- **Low Variance Models:** To ensure stability, weak learners should have **low variance**.

Conclusion: A good weak learner is **simple**, has **high bias**, and **low variance**.

Tree Stumps as Ideal Weak Learners:

- Tree stumps are decision trees with only one split (depth $= 1$).
- They are inherently high-bias but low-variance models, making them ideal for AdaBoost.

Benefits of Tree Stumps in AdaBoost

Advantages of Tree Stumps:

- **Low Variance:** Stable models reduce fluctuations that could destabilize the ensemble.
- **Fast Training:** Tree stumps are lightweight and quick to train.
- **Focus on Hard Examples:** Tree stumps naturally adjust their splits to emphasize difficult-to-classify points.
- **Reduced Overfitting:** Simple models are less prone to memorizing noise.

Key Insight: The combination of many tree stumps, each focusing on different parts of the data, allows AdaBoost to achieve remarkable accuracy while maintaining model stability.

Bias Reduction in AdaBoost

Bias Analysis in AdaBoost

- Recall the AdaBoost model:

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

- Since each weak learner $h_m(x)$ is a high-bias model, its individual prediction has large bias.
- However, as more weak learners are added, the ensemble learns to correct these errors, reducing the overall bias.
- In the limit (assuming infinite iterations and perfect weak learners):

$$F(x) = \log \frac{P(Y = 1 \mid x)}{P(Y = -1 \mid x)}$$

- This shows that AdaBoost models the "log odds" — an optimal decision boundary under the exponential loss.
- Consequently, AdaBoost effectively minimizes bias as more weak learners are combined.

Variance in AdaBoost

Variance Analysis in AdaBoost

- Although AdaBoost reduces bias effectively, its impact on variance is more complex.
- Recall that AdaBoost emphasizes difficult-to-classify points by assigning them higher weights.
- This focus on hard points may increase the model's sensitivity to noise, potentially raising variance.
- In practice:
  - For small to moderate iterations, AdaBoost effectively reduces both **bias** and **variance**.
  - For excessively large iterations, AdaBoost may start to **overfit** by placing too much emphasis on noisy points, causing variance to increase.
- Careful tuning (e.g., limiting the number of weak learners) is essential to balance bias reduction and variance control.

Extensions of AdaBoost

Extensions of AdaBoost:

- **AdaBoost.M1** – An extension of AdaBoost designed for **multiclass classification** tasks.
  - Extends the original AdaBoost algorithm to handle more than two classes.
  - Weak learners are trained in the same sequential manner, with specialized techniques for multiclass predictions.
- **AdaBoost.R2** – An extension of AdaBoost for **regression** tasks.
  - Uses weak regressors (e.g., shallow regression trees) instead of classifiers.
  - Adopts a modified loss function suited for continuous output values.

Coding Sample Part 2

Summary

Summary of Random Forest

Key Concepts:

- Random Forest combines multiple decision trees using **bagging** to improve stability and accuracy.
- Each tree is trained on a **bootstrap sample** with random feature selection at each split.

Key Advantages:

- Strong resistance to **overfitting**.
- Provides **feature importance** measures.
- Capable of handling **missing values** and works well with large datasets.

Key Limitation:

- Random Forest predictions can be less interpretable compared to individual trees.

Summary of Boosting and AdaBoost

General Boosting Concept:

- Boosting combines multiple weak learners in a **sequential** manner, where each learner corrects mistakes from the previous one.
- Focuses on **difficult points** by adjusting sample weights in each iteration.

AdaBoost:

- Uses the **exponential loss** function to prioritize misclassified points.
- Weight update follows:

$$w_{i,m+1} = w_{im} e^{-\alpha_m y_i h_m(x_i)}$$

- $\alpha_m = \frac{1}{2} \ln \left(\frac{1-\text{err}_m}{\text{err}_m} \right)$ controls each weak learner's influence.

Key Insights:

- AdaBoost effectively reduces **bias**, but may increase **variance** if overfitted.
- **Tree stumps** are commonly used as weak learners in AdaBoost for their stability and efficiency.

Thank You!

Questions are welcome.